

Donner des instructions

<https://github.com/heig-vd-progim-course/heig-vd-progim1-course>

[Support de cours](#) · [Présentation \(web\)](#) · [Présentation \(PDF\)](#)

V. Guidoux, avec l'aide de Claude.

Ce travail est sous licence [CC BY-SA 4.0](#).

Bienvenue en Programmation 1

Retrouvez plus de détails dans le support de cours

Cette présentation est un résumé du support de cours. Pour plus de détails, consultez le [support de cours](#).

Qui je suis

Vincent Guidoux, ingénieur logiciel et chargé de cours.

- Formation en informatique à la HEIG-VD, après un parcours qui n'a pas été linéaire.
- Développement web et logiciel, en indépendant et en entreprise.
- J'enseigne ici les technologies web et la programmation.

Je ne suis pas un cas particulier : la plupart des gens qui font ce métier ont commencé par ne rien y comprendre.

Mon objectif pour ce cours

Que vous sachiez décrire précisément ce que vous voulez qu'une machine fasse.

Le langage, la syntaxe, les outils : tout cela s'apprend et se remplace. La capacité à poser un problème clairement, elle, vous servira toute votre vie professionnelle, y compris ailleurs qu'en informatique.

Si quelque chose ne va pas dans ce cours, dites-le moi. Je préfère une critique en semaine 3 qu'un formulaire d'évaluation en semaine 14.

Comment me contacter

- En personne, pendant ou après la séance.
- Par courriel.
- Sur le canal Teams du cours, de préférence au message privé.

Une question posée en public profite à tout le monde et une réponse peut venir d'une autre personne que moi.

Il n'y a pas de question bête. Je suis payé pour y répondre.

Objectifs de cette séance

À la fin de cette séance, vous devriez être capable de :

- expliquer pourquoi une instruction ambiguë produit un résultat imprévisible ;
- lire et dessiner un diagramme d'activité simple ;
- représenter les règles d'un jeu sous forme de diagramme ;
- nommer deux techniques d'apprentissage efficaces ;
- avoir configuré un outil de limitation des sollicitations.



Objectifs de l'unité d'enseignement

À la fin du semestre, vous devriez être capable de :

- décrire un problème réel sous forme d'actions, de choix et de répétitions ;
- représenter cette description en diagramme et en pseudo-code ;
- traduire cette description en un programme Java correct et lisible ;
- tester et déboguer un programme simple ;
- expliquer ce que fait votre propre code et pourquoi.

Le dernier point n'est pas décoratif

Expliquer son propre code est le critère qui décide de la note du mini-projet.

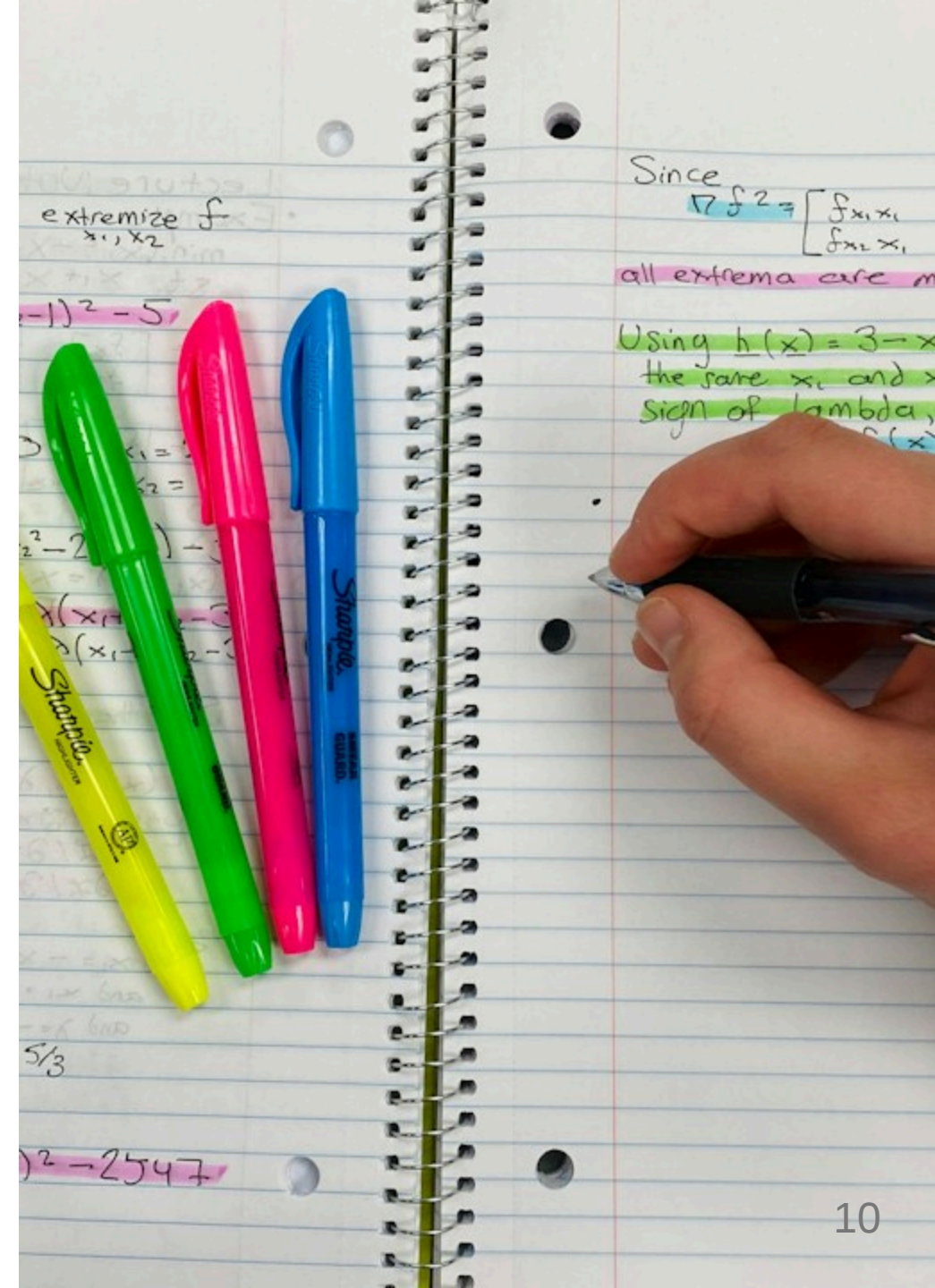
Un programme qui fonctionne et que vous ne savez pas expliquer ne vous sert à rien, ni ici, ni plus tard.

Organisation d'une séance

Quatre périodes, découpées en :

- des moments de théorie courts, pour le cadre et le vocabulaire ;
- des activités, souvent à deux, pour manipuler avant de formaliser ;
- de la pratique individuelle, pendant laquelle je passe dans les rangs.

Le support écrit sur le site du cours est la référence. Ces diapositives sont un résumé : elles ne suffisent pas à réviser.



Le semestre en deux moitiés

Séances 01 à 07 - comprendre

De l'instruction donnée à une personne jusqu'au programme Java. Une évaluation écrite à la séance 07.

Séances 08 à 12 - faire

Un mini-projet que vous choisissiez, encadré par des jalons, avec une présentation orale à la dernière séance.

Un fil rouge

Un même problème traverse les séances 03 à 09 :

Calculer la moyenne des notes d'un groupe de personnes et dire qui a réussi.

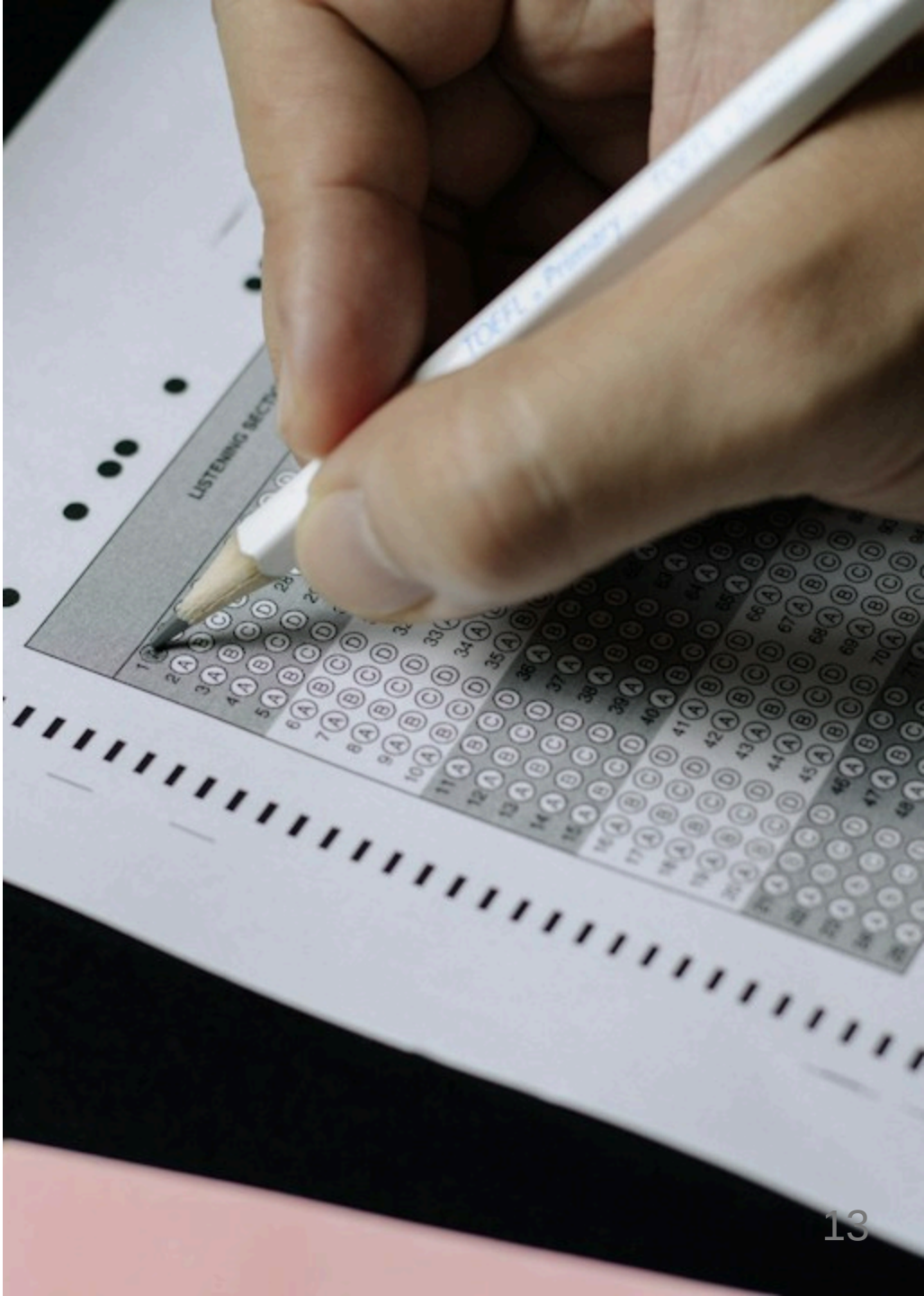
Assez simple pour être posé sans rien savoir de Java. Assez riche pour occuper successivement les variables, les conditions, les boucles, les fonctions et les tableaux.

À chaque séance, on le reprend avec l'outil du jour et on réécrit la solution précédente.

Modalités d'évaluation

Deux parties, de poids égal :

Partie	Quand	Poids
Évaluation écrite	Séance 07	50%
Mini-projet	Séances 08 à 12	50%

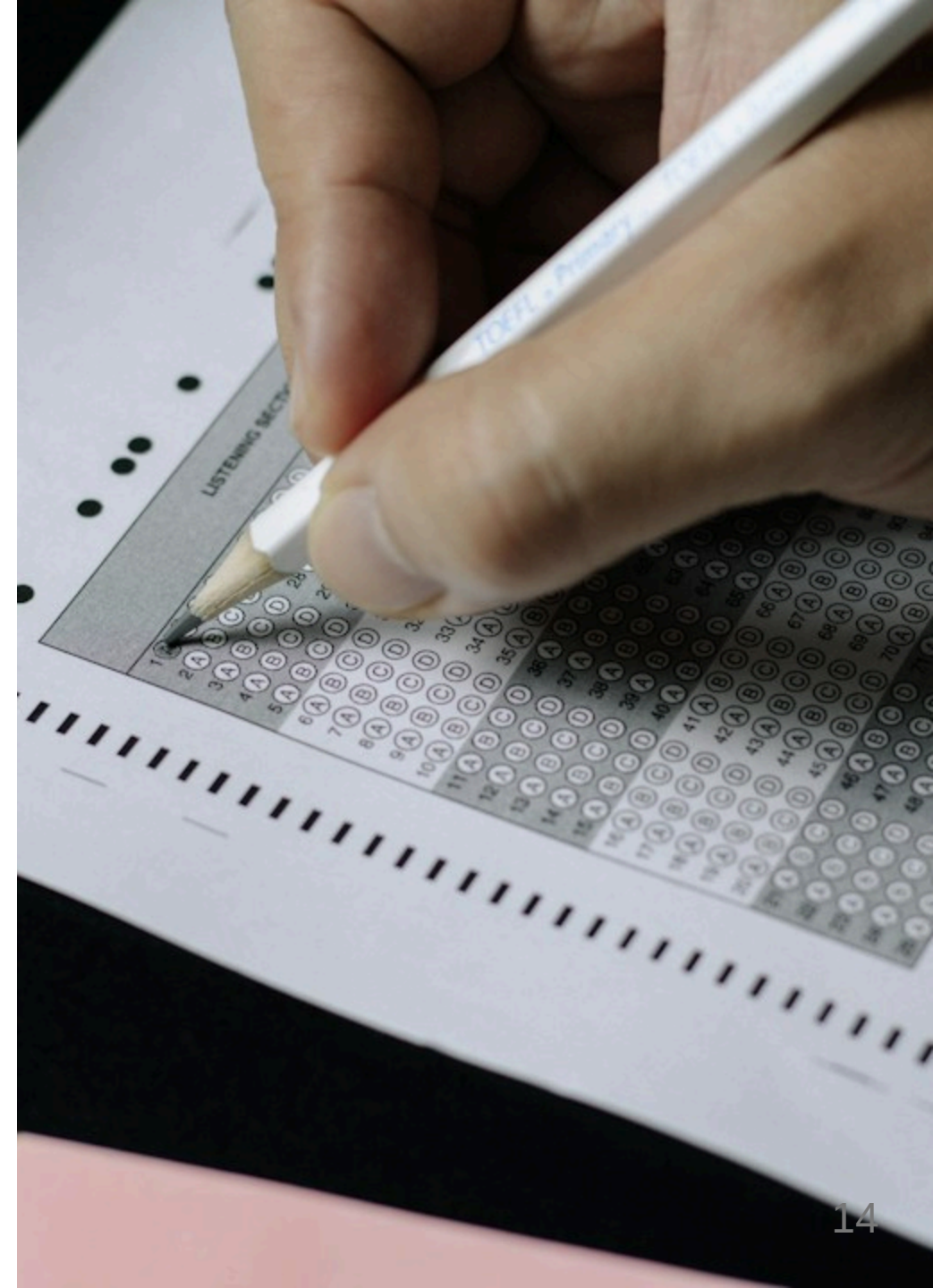


Partie 1 - Évaluation écrite (50%)

- Au milieu du semestre, en séance 07.
- Papier et crayon, sans ordinateur, sans documentation, sans IA.
- Deux périodes.

Ce qui est évalué : lire du code et dire ce qu'il produit, repérer une erreur, traduire un énoncé en diagramme, écrire un court fragment de code.

Ce qui ne l'est pas : la syntaxe au point-virgule près.



Partie 2 - Mini-projet (50%)

Un petit programme qui résout un problème que vous avez réellement.

- Vous choisissez le sujet.
- En ligne de commande, sans interface graphique ni base de données.
- Réalisé pendant les séances 08 à 12, encadré par cinq jalons.
- Évalué par une présentation orale de dix minutes.



Les jalons

Jalon	Séance	Ce que vous rendez
J1	08	Le problème choisi, entrées et sorties
J2	09	Le diagramme d'activité, dessiné à la main
J3	10	Un programme qui compile
J4	11	Le cas normal traité, cas manquants listés
J5	12	Rendu final et présentation orale

Les jalons sont **validés ou non validés**, jamais notés.

Pourquoi des jalons

Parce qu'un projet de programmation commencé la veille ne se termine pas.

Ce n'est pas une question de sérieux : c'est que face à un écran vide, on ne sait pas par où commencer, et ce moment-là prend des heures.

La présentation orale

Dix minutes, questions comprises. Ce qui est évalué :

Critère	Poids
Le problème est posé et le choix justifié	15%
La conception est expliquée	25%
Le code est compris ligne à ligne	35%
Les limites et bogues connus sont énoncés	15%
Clarté et gestion du temps	10%

Dire *"je ne sais pas expliquer cette partie"* coûte moins cher qu'inventer.

Comment se préparer

Il n'y a pas de secret et il n'y a pas de raccourci :

- refaire les exercices sans regarder la solution ;
- travailler vingt minutes, trois fois par semaine ;
- écrire du code soi-même, même mauvais.

Soixante minutes par semaine en dehors du cours suffisent à réussir. Sans elles, les quatre périodes ne suffisent pas.

Le code s'écrit en anglais

Le cours est en français. Le code est en anglais : noms de variables, de fonctions et de classes. Les commentaires restent en français.

Ce n'est pas un caprice : la documentation, les messages d'erreur et les réponses que vous trouverez sont en anglais.

Si l'anglais est une barrière pour vous, dites-le moi.

"Et ChatGPT dans tout ça ?"

"Et ChatGPT dans tout ça ?" (1/3)

Vous pouvez utiliser ces outils, sauf pendant les évaluations.

Vous les utiliserez dans votre travail ; vous en priver ici serait absurde.

Mais soyons précis sur ce qui se passe quand on les utilise en première année.

"Et ChatGPT dans tout ça ?" (2/3)

Ces outils produisent du code qui **a l'air correct**.

En première année, vous n'avez pas encore les moyens de distinguer du code correct d'un code qui en a l'air.

Vous rendez alors du code que vous ne pouvez pas déboguer, dans un langage que vous ne connaissez pas, pour un problème que vous n'avez pas analysé.

Observable chaque année : on avance plus vite quatre semaines, puis on se bloque complètement.

"Et ChatGPT dans tout ça ?" (3/3)

Deux obligations en contrepartie de la liberté :

1. Indiquer dans vos rendus **quand, pourquoi et comment** vous avez utilisé une aide extérieure.
2. Être capable **d'expliquer chaque ligne** de ce que vous rendez.

Une explication qui ne tient pas à l'oral mène au même endroit qu'une tricherie découverte : la note 1 pour la partie concernée.

"J'ai fini, je peux partir ?"

Oui. Mais d'abord, demandez-vous si vous sauriez réexpliquer ce que vous venez de faire à quelqu'un d'autre.

- Si non, vous n'avez pas fini.
- Si oui, allez aider une personne qui bloque. C'est la manière la plus efficace de consolider ce que vous venez d'apprendre.

Le but n'est pas de finir vite.

Activité 1 - Donner des instructions

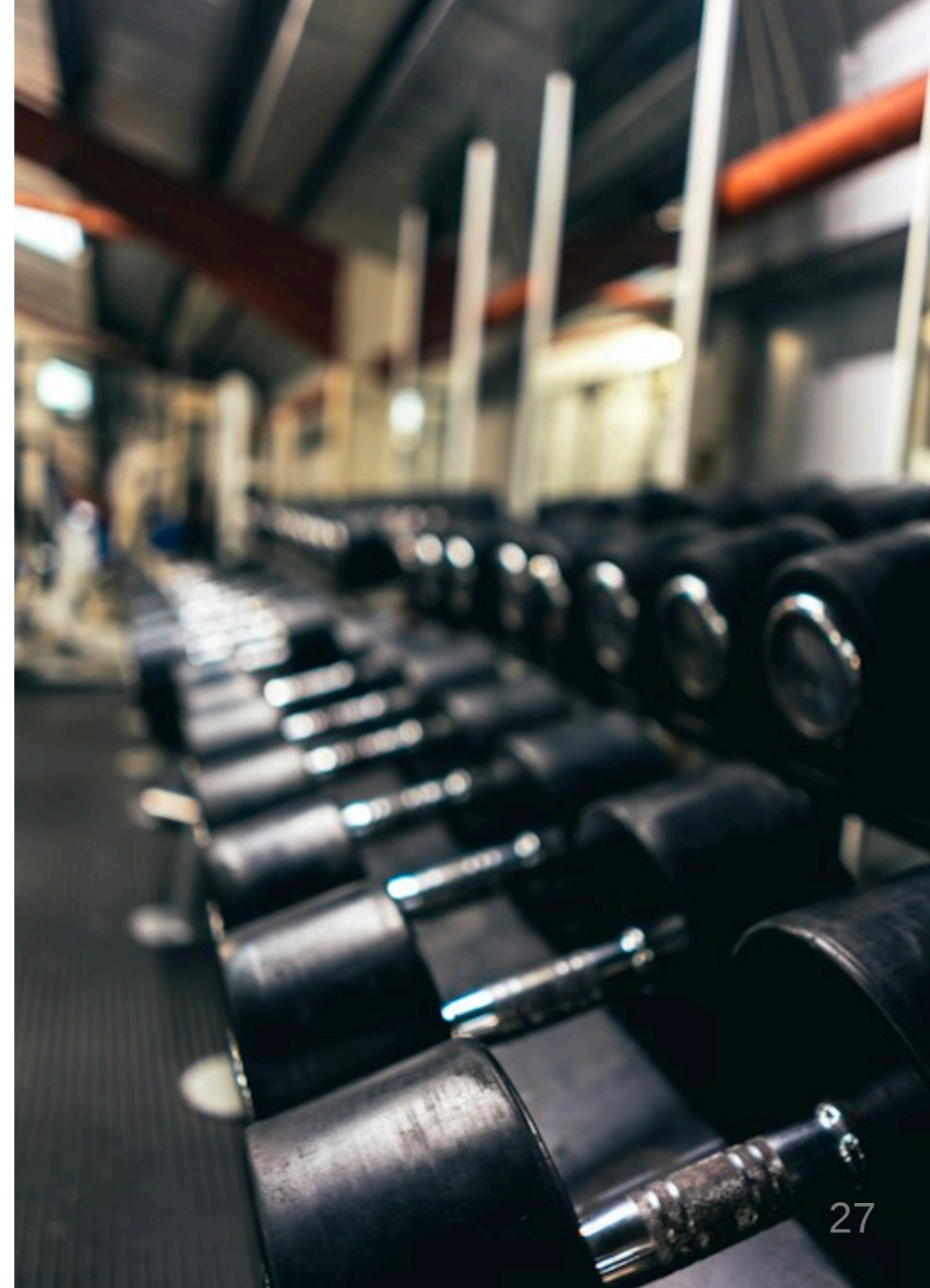
On va observer ce qui se passe quand on donne des instructions à quelqu'un qui ne peut pas deviner ce qu'on a en tête.

Activité 1 - Comment ça marche

Par deux, dos à dos.

- Une personne **voit une figure** et la décrit à voix haute.
- L'autre **dessine** exactement ce qu'elle entend, sans interpréter.
- Manche 1 : aucune question autorisée.
- Manche 2 : on inverse les rôles, questions fermées autorisées.

Six minutes par manche. On compare ensuite les deux feuilles.



Activité 1 - Ce qu'on en retient

- Sans **vocabulaire partagé**, une instruction ne veut rien dire.
- L'**ordre** des instructions change le résultat.
- Une instruction dépend d'un **état** que les deux parties doivent maintenir à l'identique.
- Une erreur au début **se propage** jusqu'à la fin.
- Un système qui peut **signaler son état** se corrige ; un système muet, non.

Ce qu'il faut donc savoir écrire

Une description qui est :

- non ambiguë ;
- ordonnée ;
- dans un vocabulaire partagé ;
- relative à un état connu.

La question suivante : comment écrire cela pour que quelqu'un d'autre puisse le vérifier ?

Réponse : on le dessine.

Les diagrammes d'activité

UML compte quatorze types de diagrammes et des centaines de symboles.

Nous en utilisons six.

Et nous les dessinons à la main, au crayon. Un diagramme au crayon prend trente secondes et on ose le jeter ; le même fait avec un outil prend vingt minutes et on s'y accroche.

Les six symboles

Symbole	Nom	Sens
Disque noir plein	Nœud initial	Ici ça commence
Rectangle arrondi	Action	On fait quelque chose
Flèche	Transition	Puis on passe à la suite
Losange, une entrée	Décision	On pose une question
Losange, plusieurs entrées	Fusion	Les chemins se rejoignent
Disque noir cerclé	Nœud final	Ici ça se termine

Deux règles de lecture

1. Une flèche qui sort d'une décision porte une **condition** écrite entre crochets :
[il pleut] , [sinon] .
2. Depuis une décision, **un seul** chemin est emprunté.

Erreur la plus fréquente : oublier le chemin pour le cas [sinon] .

Séquence, sélection, itération

Tout programme, dans n'importe quel langage, est construit avec trois structures et rien d'autre :

- la **séquence** : faire ceci, puis cela ;
- la **sélection** : selon la situation, faire ceci ou cela ;
- l'**itération** : répéter tant qu'une condition tient.

Ce n'est pas une simplification pédagogique, c'est un théorème.

Sur une boucle, vérifiez toujours

1. La condition de sortie **existe**.
2. Quelque chose **change** à chaque tour.

Sans le point 2, la condition de sortie ne devient jamais vraie et le programme tourne indéfiniment.

C'est la boucle infinie, l'une des deux erreurs que vous ferez le plus souvent cette année.

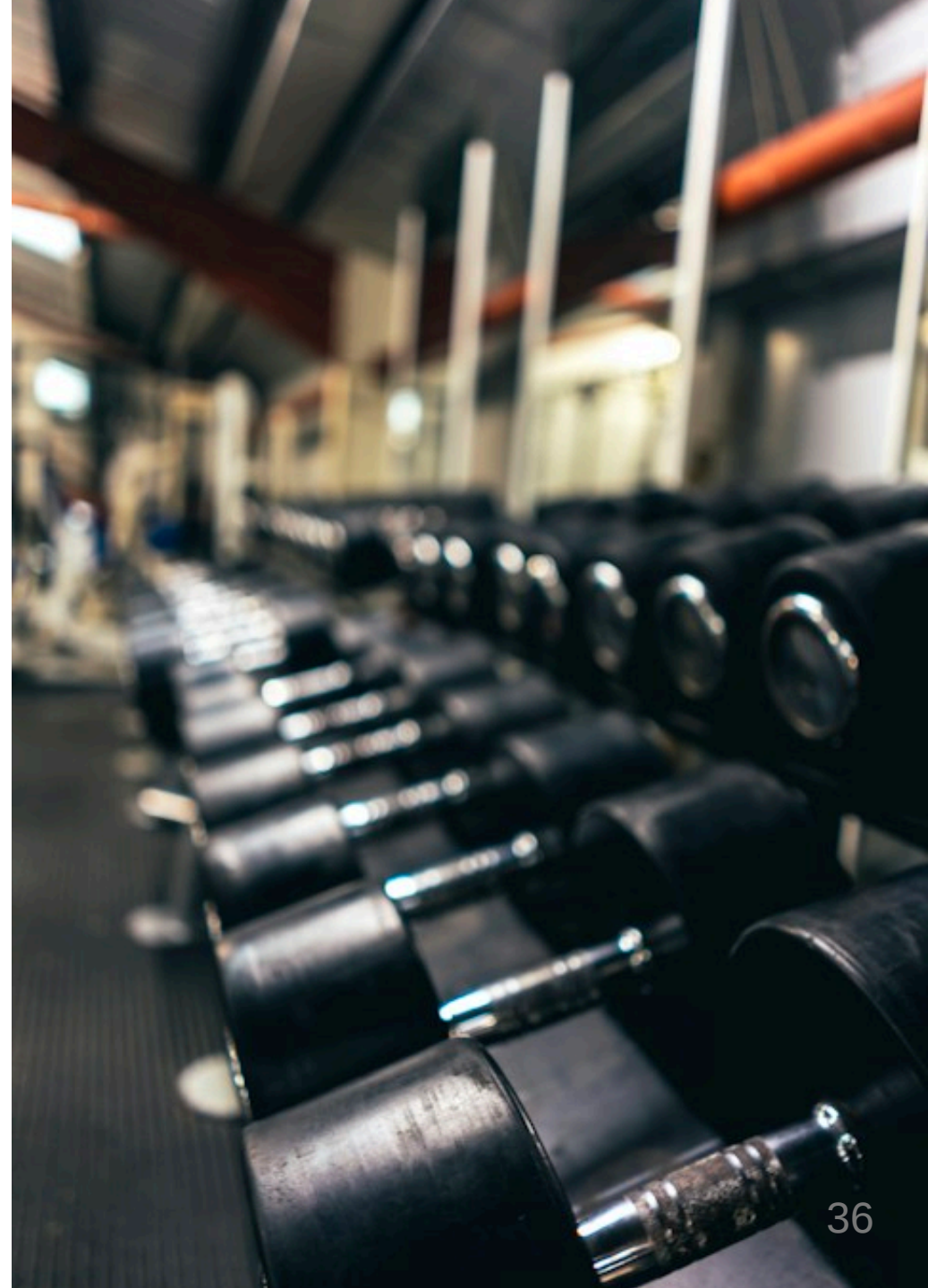
Activité 2 - Les règles du jeu

Les règles d'un jeu de société décrivent une séquence, des choix et une répétition. C'est la structure d'un programme, écrite par des gens qui ne faisaient pas d'informatique.

Activité 2 - Comment ça marche

En groupes de trois ou quatre, avec un jeu et ses règles. **On ne joue pas.**

1. Lire les règles une fois, en entier.
2. Répondre par écrit : quand la partie s'arrête-t-elle ? Quels choix a-t-on pendant son tour ?
3. Dessiner le diagramme d'activité **d'un seul tour.**
4. Entourer en rouge **tout ce que les règles ne disent pas** et que vous avez dû deviner.



Activité 2 - Restitution

Deux minutes par groupe :

1. Le nom du jeu et sa condition de fin.
2. Le diagramme.
3. **La chose la plus importante que les règles ne disaient pas.**

Ce que vous venez de faire porte un nom : une **spécification**.

Apprendre à apprendre

La programmation est une des rares matières où le travail régulier n'est pas un conseil moral mais une contrainte technique.

Trois choses qui fonctionnent

1. **Se tester plutôt que relire.** Relire donne une sensation de maîtrise qui ne correspond à rien.
2. **Espacer plutôt que grouper.** Trois fois vingt minutes valent mieux qu'une heure le dimanche soir.
3. **Écrire le code soi-même**, même mauvais. Lire une solution et savoir en produire une sont deux compétences différentes.

Une chose qui ne fonctionne pas

Travailler avec le téléphone à côté, même retourné, même silencieux.

Le coût d'une notification n'est pas les trois secondes qu'elle prend : c'est le temps de reconstruire l'état mental dans lequel vous teniez votre programme.

Ces applications sont conçues par des équipes dont c'est le métier. Perdre contre elles n'est pas un défaut de caractère.

Atelier - reprendre la main

Vingt minutes, maintenant.

Sur le téléphone : couper les notifications de tout sauf appels et messages directs.
Puis ScreenZen, ou Olauncher, ou le Mode Concentration.

Sur l'ordinateur : LeechBlock NG, SelfControl, Cold Turkey, ou le fichier `hosts` .

Objectif de sortie : un outil actif sur le téléphone, un sur l'ordinateur.

À vous de jouer !

- Terminer l'installation des outils et les utiliser au moins une fois.
- Reprendre le diagramme d'activité de votre jeu et le corriger à tête reposée.
- Relire le support de cours de la séance.
- Bloquer dans votre agenda trois créneaux de vingt minutes par semaine.



Questions

Est-ce que vous avez des questions ?

Sources

- [Illustration principale](#) par [Richard Jacobs](#) sur [Unsplash](#)
- [Illustration objectifs](#) par [Aline de Nadai](#) sur [Unsplash](#)
- [Illustration organisation](#) par [American Jael](#) sur [Unsplash](#)
- [Illustration activité](#) par [Samuel Girven](#) sur [Unsplash](#)
- [Illustration mini-projet](#) par [Alec Favale](#) sur [Unsplash](#)
- [Illustration évaluation](#) par [Nguyen Dang Hoang Nhu](#) sur [Unsplash](#)
- [Illustration à vous de jouer](#) par [Tim van Cleef](#) sur [Unsplash](#)